ALGORITMOS Y PROGRAMACIÓN II

Clase 8
Árboles binarios de búsqueda

Temas de la clase

- Generalización del uso del ABB
  - Contando ocurrencias
  - Almacenando datos asociados en cada nodo

Actividades en Máquina

ACTIVIDAD 1: Cree el archivo `Nombres.pas`

Cree un programa que cree y utilice un ABB para almacenar nombres de persona.

- Implemente un módulo cargarArbol que reciba un ABB y lo llene con nombres, hasta la aparición del nombre 'ZZZ'.
- Implemente un módulo que reciba un ABB y un nombre (como string) e imprima si dicho nombre está o no en el árbol.
- Escriba un programa que cree un ABB de nombres, invoque a los dos módulos implementados y finalmente imprima todos los nombres almacenados en el árbol y luego imprima solo los nombres que se encuentran entre otros dos nombres leídos por teclado.

¿Puede reutilizar los módulos implementados en la clase anterior?

Árboles de registros

El Nodo del árbol deberá tener el registro con todos los campos que se desean almacenar en el ABB.

```
type

        Nodo = record
                campo1, campo2, campo3 : integer;
                campo4, campo5: string;
                ...
        end;

        ABBGenerico = specialize ABB<Nodo>;
```

Actividades en Máquina

ACTIVIDAD 2: Descargue el archivo `ArbolDeClientes.pas`

Un negocio *dispone* de la información de todos sus clientes. De cada cliente se conoce: nombre, apellido, DNI y categoría (1 a 5).

- Utilice el módulo **ObtenerClientes** ya implementado por la cátedra para obtener una lista de clientes. Implemente un módulo **cargarArbol** que reciba un ABB y lo llene con los clientes de la lista que se *dispone*. Los clientes deberán quedar ordenados por DNI.
- Implemente un módulo que reciba un ABB de clientes y un DNI e imprima su categoría si es que el cliente existe dentro del árbol.
- Implemente un módulo que reciba un ABB de clientes y dos DNI, e imprima nombre y apellido de todos los clientes que están dentro del rango formado por los dos DNIs recibidos por parámetro.
- Implemente un módulo que reciba un ABB de clientes y una categoría, e imprima todos los datos de los clientes que tienen dicha categoría.
- Escriba un programa que instancie un ABB de clientes e invoque a los cuatro módulos implementados.

Actividades en Máquina

ACTIVIDAD 3: Descargar el archivo `ListaDePrecios.pas`

Un local de artículos deportivos dispone de la lista de precios de todos los productos que vende. De cada producto se conoce código único, descripción y precio.

- Utilice el módulo **ObtenerListaDePrecios** ya implementado por la cátedra para obtener una lista de productos. A partir de dicha lista arme un ABB con todos los productos que vende el local, ordenado por código único del producto.
- Implemente un módulo que reciba un ABB de productos y un código único e informe el precio del producto si es que el producto existe dentro del árbol.
- Implemente un módulo que reciba un ABB de productos e imprima la descripción del producto más barato y del más caro.
- Escriba un programa que instancie un ABB de clientes e invoque a los tres módulos implementados.

Actividades en Máquina

ACTIVIDAD 4: Modifique la actividad anterior.

- Implemente un módulo que reciba un ABB de productos y una lista con códigos de productos (que representa una lista de compra) y retorne el precio total de la compra. Utilice el módulo **ObtenerListaDeCompra** ya implementado por la cátedra para obtener una lista de compras.
- Modifique el programa principal para cargar una lista de compra e invoque al módulo implementado en esta actividad imprimiendo el total de la compra.

Actividades en Máquina

ACTIVIDAD 5: Modifique la actividad anterior

Modifique la actividad anterior para incrementar el precio de los productos del local deportivo.

- Implemente un módulo que reciba un ABB de productos y permita aumentar un 10% el precio de todos los productos.
- Modifique el programa principal invocando al módulo implementado en esta actividad imprimiendo el precio de los productos antes y después del aumento.

Para modificar un registro que está almacenado dentro del árbol utilice:

```
prod:= arbol.current();
prod.precio:= prod.precio * 1.1;
arbol.setCurrent(prod);
```

Problema

Supongamos que durante el dictado de una clase, el docente a cargo quiere almacenar en un ABB a los estudiantes que participan en clase respondiendo las preguntas que realiza el docente.

Cada vez que un estudiante responde una pregunta, el docente lo agrega al ABB. En el ABB solo se almacena el número de alumno.

¿Qué sucede con los alumnos que no participan en clase?

¿Qué sucede con los alumnos que responden dos o más veces?

Contando ocurrencias

Vimos que si el dato se repite, el ABB los almacena repetidos en nodos distintos.

Lo ideal sería almacenar un único nodo por dato, contando ocurrencias de repetición. Es decir, los datos dentro del árbol no se repiten y cada nodo debería llevar la contabilidad del dato que almacena.

Necesitamos que el nodo del árbol lleve dicha contabilidad:

```
type

        Nodo = record
                clave: TIPO_DE_DATO;
                campo1, campo2, campo3: integer;
                campo4, campo5: string;
                ...
                ocurrencias: integer;
        end;
```

Uno de los campos debe cumplir la función de clave única, es decir, el árbol no puede tener dos o más nodos con claves repetidas.

El propio nodo lleva la contabilidad de ocurrencias de la clave que almacena

Contando ocurrencias

```
type
  Nodo = record
         ocurrencias: integer;
         numeroAlumno: integer;
  end;
  ABBParticipaciones = specializa ABB<Nodo>;

  var a: ABBParticipaciones;
  begin
  a:= ABBParticipaciones.create();

  agregarNodo(a, 1101);

  ...

  agregarNodo(a, 1101);
  end.
```

La primera vez la clave no existe, entonces se agrega un nuevo nodo al árbol

Contando ocurrencias

```
type
  Nodo = record
         ocurrencias: integer;
         numeroAlumno: integer;
  end;
  ABBParticipaciones = specializa ABB<Nodo>;

  var a: ABBParticipaciones;
  begin
  a:= ABBParticipaciones.create();

  agregarNodo(a, 1101);

  ...

  agregarNodo(a, 1101);
  end.
```

La clave ya existe!!!! Deberíamos incrementar el contador de ocurrencias del nodo.

Contando ocurrencias

Mientras buscamos donde insertar el nuevo nodo, deberíamos revisar si la clave existe. Si existe incrementar la ocurrencia.

```
procedure agregarNodo(var a: ABBParticipaciones; num: integer);
  var n: Nodo;
  begin
  if a.isEmpty() then    begin
      n.ocurrencias:= 1;    n.numeroAlumno:= num;
      a.insertCurrent(n);
      end
  else
      if a.current().numeroAlumno = num then  begin
          n:= a.current();
          n.ocurrencias:= n.ocurrencias + 1;
          a.setCurrent(n);
          end
      else
          { Buscar a izquierda o a derecha según corresponda }
          ...
  end;
```

Actividades en Máquina

ACTIVIDAD 6: Modifique la actividad anterior

Modifique el programa de la actividad anterior para que el local lleve la contabilidad de cuántas unidades vende de cada producto, junto con el total recaudado de cada producto.

- Implemente un módulo que simule diferentes ventas de los productos del local (invocando varias veces al módulo **ObtenerListaDeCompra**), manteniendo actualizadas las contabilidades pedidas.
- Modifique el programa principal para que invoque al módulo implementado en esta actividad y verifique el correcto funcionamiento del mismo imprimiendo toda la información almacenada.

Almacenando varios datos en un nodo

¿Cómo podríamos hacer para almacenar en el mismo nodo varios datos asociados a la misma clave?
Por ejemplo, para el mismo alumno (clave legajo) todos sus exámenes. Para el mismo jugador de fútbol (clave dni) todos sus partidos jugados, etc.

Necesitamos que el nodo del árbol almacene una lista con los detalles del dato principal:

```
type   DatoDetalle = record
                 campoA, campoB, campoC: real;
                 campoD, campoE: boolean;
                 ...
       end;
       ListaDeDatosDetalles = specialize LinkedList<DatoDetalle>;
       Nodo = record
                 clave: TIPO_DE_DATO;
                 campo1, campo2, campo3: integer;
                 campo4, campo5: string;
                 ...
                 datos: ListaDeDatosDetalles;
       end;
```

Almacenando varios datos en un nodo

Mientras buscamos donde insertar el nuevo nodo, deberíamos revisar si la clave existe. Si existe agregar el detalle a la lista del nodo.

```
procedure agregarNodo(var a: ABBConListas; dc: DatosCompletos);
  var n: Nodo;     dd: DatoDetalle;
  begin
  if a.isEmpty() then    begin
      n.datos:= ListaDeDatosDetalles.create() ;
      dd:= armarRegistroDetalle(dc);
      n.datos.add(dd);
      n.clave:= dc.campo3;
      n.campo1:= dc.campo1;
      n.campo2:= dc.campo2;
      ...
      a.insertCurrent(n);
      end
  else
      if a.current().numeroAlumno = num then  beg
          ...
  end;
```

Del registro con todos los datos, se arma un regstro con los datos detalles

Se debe almacenar como clave el campo que va a actuar como tal.
Y de ser necesario, al nodo del árbol se le copian todos los campos “únicos”.

Almacenando varios datos en un nodo

Mientras buscamos donde insertar el nuevo nodo, deberíamos revisar si la clave existe. Si existe agregar el detalle a la lista del nodo.

```
procedure agregarNodo(var a: ABBConListas; dc: DatosCompletos);
  var n: Nodo;     dd: DatoDetalle;
  begin
  if a.isEmpty() then    begin
      ...
      end
  else
      if a.current().clave = dc.campo3 then  begin
          dd:= armarRegistroDetalle(dc);
          a.current().datos.add(dd);
          end
      else
          { Buscar a izquierda o a derecha según corresponda }
          ...
  end;
```

Solo se arma un regstro con los datos detalles y se lo agrega a la lista.

Actividades en Máquina

ACTIVIDAD 7: Descargue el archivo `BilleteraVirtual.pas`

Un banco *dispone* de la información de las billeteras virtuales de sus clientes. De cada Billetera Virtual se conoce: CVU, saldo actual y los movimientos realizados. Los movimientos registran el CVU de destino/origen y el monto transferido (un valor positivo si se recibió dinero, o un valor negativo si se envió.

- Utilice el módulo **ObtenerTransacciones** ya implementado por la cátedra para obtener una lista de transacciones entre billeteras. Implemente un módulo **cargarArbol** que reciba un ABB y lo llene con los la información de la lista que se *dispone*. Las billeteras deberán quedar ordenados por CVU.
- Finalmente imprima el árbol completo para comprobar el correcto funcionamiento del programa.

Actividades en Máquina

ACTIVIDAD 8: Modifique la actividad anterior

- Implemente un módulo que reciba un CVU y un ABB previamente cargado e imprima el movimiento negativo más chico, el movimiento negativo más grande, el movimiento positivo más chico, el movimiento positivo más grande y el valor promedio entre todos los movimientos de la billetera con el CVU recibido.
- Implemente un módulo que reciba un ABB previamente cargado y devuelva la cantidad de billeteras con saldo actual negativo.
- Modifique el programa principal para invocar a los módulos implementados en esta actividad y comprobar el correcto funcionamiento de los mismos.